

Lexical Analysis – part 2

Converting Regular Expression to DFA

We may convert a regular expression into a DFA without creating a NFA first

First we augment the given regular expression by concatenating it with a special symbol #

$r \rightarrow (r)\#$: augmented regular expression

Then, we create a syntax tree for this augmented regular expression

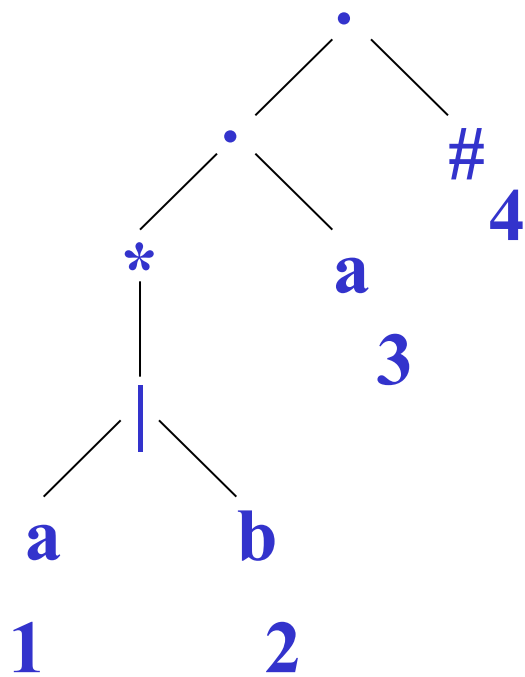
In this syntax tree

All **alphabet symbols, #, and the empty string** in the augmented regular expression will be on the **leaves**

All **inner nodes** will be the **operators**

Then each **alphabet symbol and #** will be numbered (**position numbers**).

$(a|b)^*a \sqsubseteq (a|b)^*a\#$: augmented regular expression



Syntax tree of $(a|b)^*a\#$

each symbol is numbered

each symbol is at a leaf

inner nodes are operators

followpos()

for the positions assigned to leaves compute:

followpos(i) : is the set of positions which can *follow* the **position *i*** in the strings generated by the augmented regular expression.

followpos()

for the positions assigned to leaves compute:

followpos(i) : is the set of positions which can *follow* the **position *i*** in the strings generated by the augmented regular expression.

For example (a | b) * a #
 1 2 3 4

followpos(1) =

followpos()

for the positions (positions assigned to leaves) compute

followpos(i) : is the set of positions which can *follow* the **position i** in the strings generated by the augmented regular expression.

For example (a | b) * a #
 1 2 3 4

followpos(1) = {1, 2, 3}

followpos(2) =

followpos()

for the positions (positions assigned to leaves) compute

followpos(i) : is the set of positions which can *follow* the **position i** in the strings generated by the augmented regular expression.

For example (a | b) * a #
 1 2 3 4

followpos(1) = {1, 2, 3}

followpos(2) = {1, 2, 3}

followpos(3) =

followpos(4) =

followpos()

for the positions (positions assigned to leaves) compute

followpos(i) : is the set of positions which can *follow* the **position i** in the strings generated by the augmented regular expression.

For example (a | b) * a #
 1 2 3 4

followpos(1) = {1, 2, 3}

followpos(2) = {1, 2, 3}

followpos(3) = {4}

followpos(4) = {}

To evaluate followpos()

we need **three** more functions to be defined for the nodes (not just for leaves) of the syntax tree.

firstpos(n) = the set of the positions of the **first symbols** of strings generated by the **sub-expression rooted by node n**.

To evaluate followpos()

we need **three** more functions to be defined for the nodes (not just for leaves) of the syntax tree.

firstpos(n) = the set of the positions of the **first symbols** of strings generated by the **sub-expression rooted by node n**.

lastpos(n) = the set of the positions of the **last symbols** of strings generated by the **sub-expression rooted by node n**.

To evaluate followpos()

we need **three** more functions to be defined for the nodes (not just for leaves) of the syntax tree.

firstpos(n) = the set of the positions of the **first symbols** of strings generated by the **sub-expression rooted by node n**.

lastpos(n) = the set of the positions of the **last symbols** of strings generated by the **sub-expression rooted by node n**.

nullable(n) = *true* if the **empty string** is a member of strings generated by the **sub-expression rooted by n**,
false otherwise

Algorithm: Regular expression to equivalent DFA

Put **firstpos(root)** into the states of DFA as an **unmarked state**.

This is the **start state** of DFA

while (there is an **unmarked state S** in the states of DFA) do

mark S

 for each **input symbol a** do

 let $s_1, \dots, s_n$ are positions in **S** and symbols in those positions are **a**

$S' \sqsubset \text{followpos}(s_1) \cup \dots \cup \text{followpos}(s_n)$

move (S, a) $\sqsubset$ **S'**

 if (**S'** is not empty and not in the states of DFA)

 put **S'** into the states of DFA as an **unmarked state**.

The **accepting** states of DFA are **all states** containing the position of #

Example - (a | b) * a

1 2 3 4

followpos(1)={1,2,3}

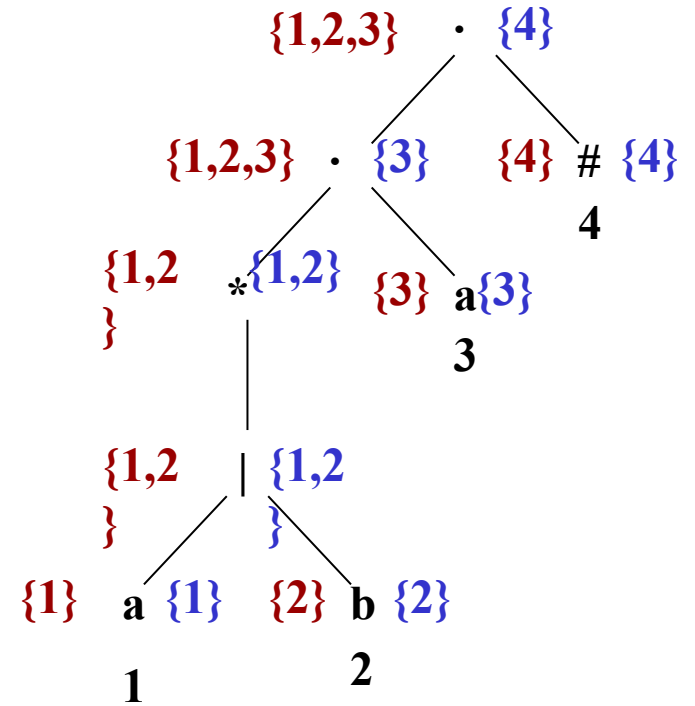
followpos(2)={1,2,3}

followpos(3)={4}

followpos(4)={}

$S_1 = \text{firstpos}(\text{root}) =$

start state: S_1



red – firstpos
blue – lastpos

Example - (a | b) * a #
1 2 3 4

followpos(1)={1,2,3} followpos(2)={1,2,3}

followpos(3)={4} followpos(4)={}

$S_1 = \text{firstpos}(\text{root}) = \{1,2,3\} = \text{Start State}$

Example - (a | b) * a #
1 2 3 4

followpos(1)={1,2,3} **followpos(2)={1,2,3}**

followpos(3)={4} **followpos(4)={}**

$S_1 = \text{firstpos}(\text{root}) = \{1,2,3\} = \text{Start State}$

$\Downarrow$ mark S_1

$\text{move}(S_1, a) =$

Example - (a | b) * a

1 2 3 4

$\text{followpos}(1) = \{1, 2, 3\}$ $\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$ $\text{followpos}(4) = \{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\} = \text{Start State}$

↓ mark S_1

$\text{move}(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_1, b) =$

Example - (a | b) * a

1 2 3 4

$\text{followpos}(1) = \{1, 2, 3\}$ $\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$ $\text{followpos}(4) = \{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\} = \text{Start State}$

↓ mark S_1

$\text{move}(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_1, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

Example - (a | b) * a

1 2 3 4

$\text{followpos}(1) = \{1, 2, 3\}$ $\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$ $\text{followpos}(4) = \{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\} = \text{Start State}$

↓ mark S_1

$\text{move}(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_1, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

↓ mark S_2

$\text{move}(S_2, a) =$

$\text{move}(S_2, b) =$

Example - (a | b) * a

1 2 3 4

$\text{followpos}(1) = \{1, 2, 3\}$ $\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$ $\text{followpos}(4) = \{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\} = \text{Start State}$

↓ mark S_1

$\text{move}(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_1, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

↓ mark S_2

$\text{move}(S_2, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_2, b) =$

Example - (a | b) * a

1 2 3 4

$\text{followpos}(1) = \{1, 2, 3\}$ $\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$ $\text{followpos}(4) = \{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\} = \text{Start State}$

↓ mark S_1

$\text{move}(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_1, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

↓ mark S_2

$\text{move}(S_2, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_2, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

Example - $(a \mid b)^* a \#$

1 2 3 4

$\text{followpos}(1)=\{1,2,3\}$ $\text{followpos}(2)=\{1,2,3\}$

$\text{followpos}(3)=\{4\}$ $\text{followpos}(4)=\{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1,2,3\} = \text{Start State}$

↓ mark S_1

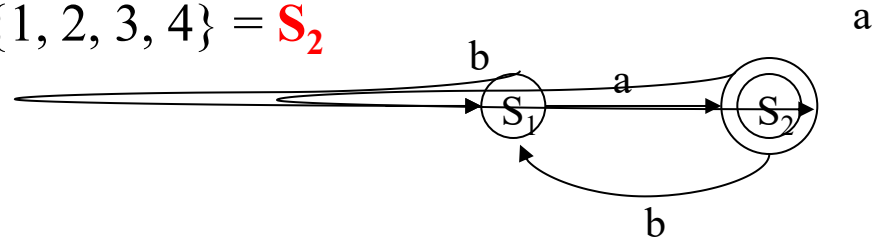
$\text{move}(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_1, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

↓ mark S_2

$\text{move}(S_2, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_2, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$



Example - (a | b) * a

1 2 3 4

$\text{followpos}(1) = \{1, 2, 3\}$ $\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$ $\text{followpos}(4) = \{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\} = \text{Start State}$

↓ mark S_1

$\text{move}(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

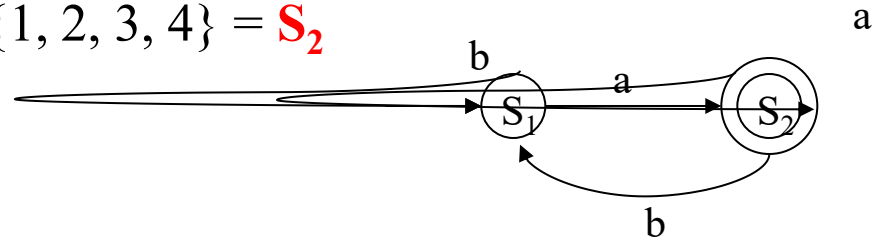
$\text{move}(S_1, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

↓ mark S_2

$\text{move}(S_2, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_2, b) = \text{followpos}(2) = \{1, 2, 3\} = S_1$

accepting states: $\{S_2\}$



To evaluate followpos()

we need **three** more functions to be defined for the nodes (not just for leaves) of the syntax tree.

firstpos(n) = the set of the positions of the **first symbols** of strings generated by the **sub-expression rooted by node n**.

lastpos(n) = the set of the positions of the **last symbols** of strings generated by the **sub-expression rooted by node n**.

nullable(n) = *true* if the **empty string** is a member of strings generated by the **sub-expression rooted by n**,
false otherwise

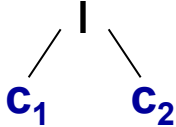
n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false		

firstpos(n) = the set of the positions of the **first symbols** of strings generated by the sub-expression rooted by node n.

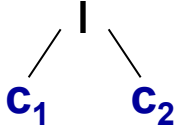
lastpos(n) = the set of the positions of the **last symbols** of strings generated by the sub-expression rooted by node n.

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	{i}	{i}
			

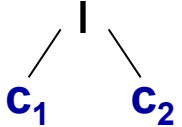
nullable(n) = *true* if the **empty string** is a member of strings generated by the sub-expression rooted by n,

false otherwise

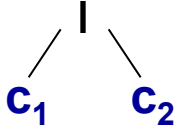
n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	{i}	{i}
	nullable(c_1) or nullable(c_2)		

nullable(n) = *true* if the **empty string** is a member of strings generated by the sub-expression rooted by n,

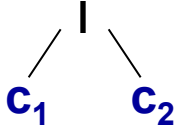
false otherwise

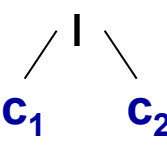
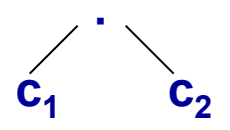
n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)		

firstpos(n) = the set of the positions of the **first symbols** of strings generated by the sub-expression rooted by node n.

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	

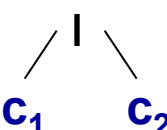
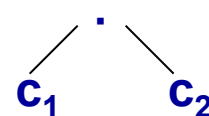
lastpos(n) = the set of the positions of the **last symbols** of strings generated by the sub-expression rooted by node n.

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
 <pre> graph TD l[l] --- c1[c1] l --- c2[c2] </pre>	$\text{nullable}(c_1)$ or $\text{nullable}(c_2)$	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$

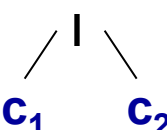
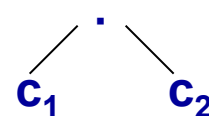
n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	{i}	{i}
	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
			

nullable(n) = *true* if the **empty string** is a member of strings generated by the sub-expression rooted by n,

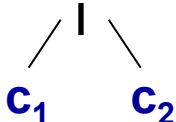
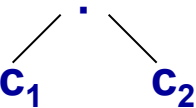
false otherwise

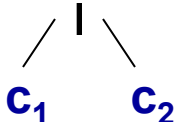
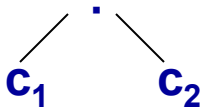
n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
	nullable(c_1) and nullable(c_2)		

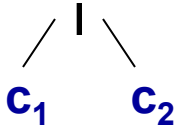
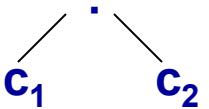
firstpos(n) = the set of the positions of the **first symbols** of strings generated by the sub-expression rooted by node n.

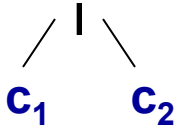
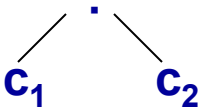
n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) firstpos(c_1) $\cup$ firstpos(c_2) else firstpos(c_1)	

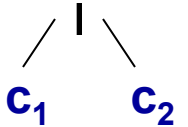
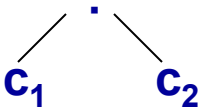
lastpos(n) = the set of the positions of the **last symbols** of strings generated by the sub-expression rooted by node n.

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
 <pre> graph TD n1[] --- c1[c1] n1 --- c2[c2] </pre>	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
 <pre> graph TD n2[] . c1[c1] n2 . c2[c2] </pre>	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) firstpos(c_1) $\cup$ firstpos(c_2) else firstpos(c_1)	if (nullable(c_2)) lastpos(c_1) $\cup$ lastpos(c_2) else lastpos(c_2)

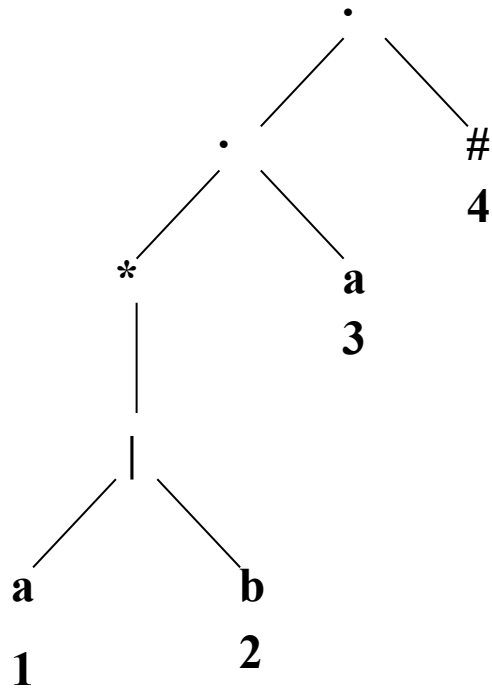
n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) firstpos(c_1) $\cup$ firstpos(c_2) else firstpos(c_1)	if (nullable(c_2)) lastpos(c_1) $\cup$ lastpos(c_2) else lastpos(c_2)
			

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) firstpos(c_1) $\cup$ firstpos(c_2) else firstpos(c_1)	if (nullable(c_2)) lastpos(c_1) $\cup$ lastpos(c_2) else lastpos(c_2)
	true		

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)	$\text{firstpos}(c_1) \cup$ $\text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup$ $\text{lastpos}(c_2)$
	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) $\text{firstpos}(c_1) \cup$ $\text{firstpos}(c_2)$ else $\text{firstpos}(c_1)$	if (nullable(c_2)) $\text{lastpos}(c_1) \cup$ $\text{lastpos}(c_2)$ else $\text{lastpos}(c_2)$
	true	$\text{firstpos}(c_1)$	

n	nullable(n)	firstpos(n)	lastpos(n)
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	$\{i\}$	$\{i\}$
	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) firstpos(c_1) $\cup$ firstpos(c_2) else firstpos(c_1)	if (nullable(c_2)) lastpos(c_1) $\cup$ lastpos(c_2) else lastpos(c_2)
	true	firstpos(c_1)	lastpos(c_1)

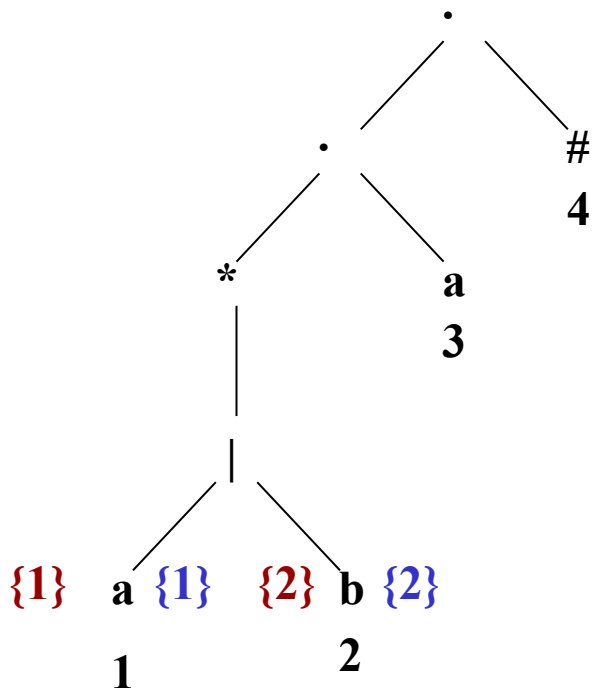
Example - $(a \mid b)^* a \#$



Example - $(a \mid b)^* a \#$

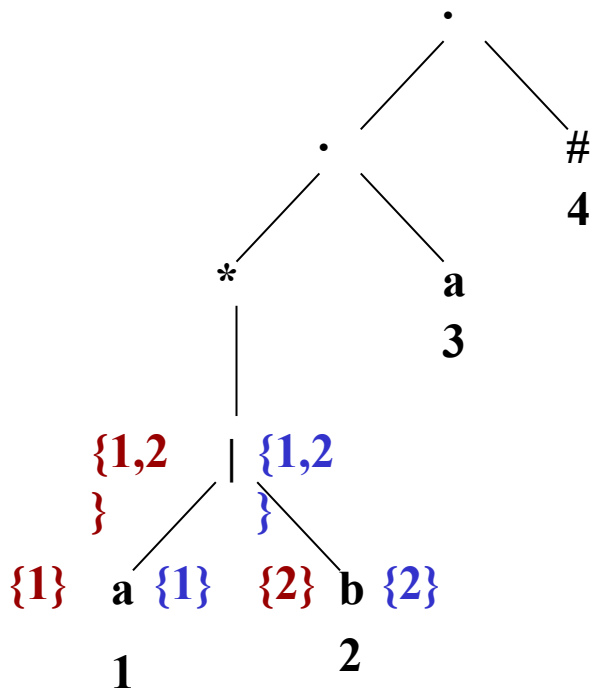
red – firstpos

blue – lastpos



Example - $(a \mid b)^* a \#$

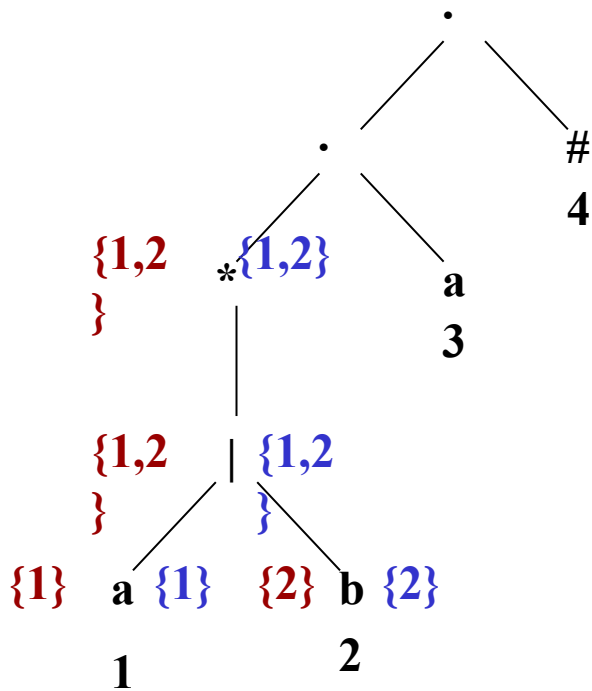
red – firstpos
blue – lastpos



Example - $(a \mid b)^* a \#$

red – firstpos

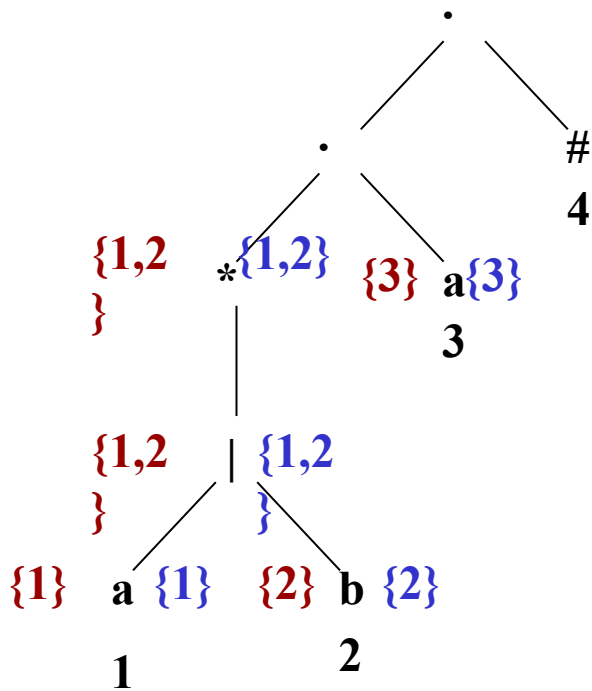
blue – lastpos



Example - $(a \mid b)^* a \#$

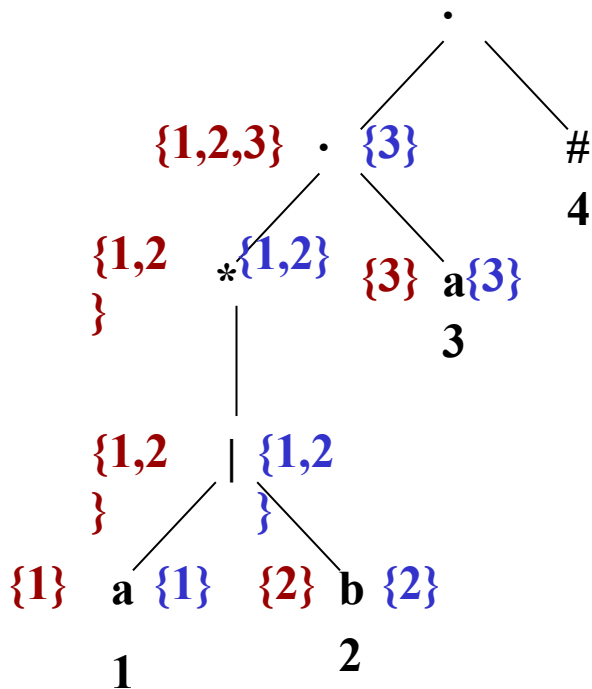
red – firstpos

blue – lastpos

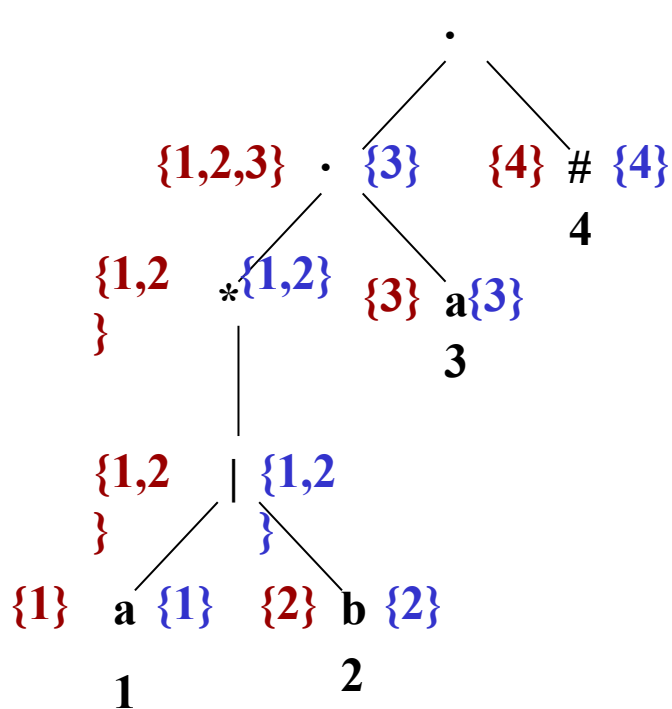


Example - $(a \mid b)^* a \#$

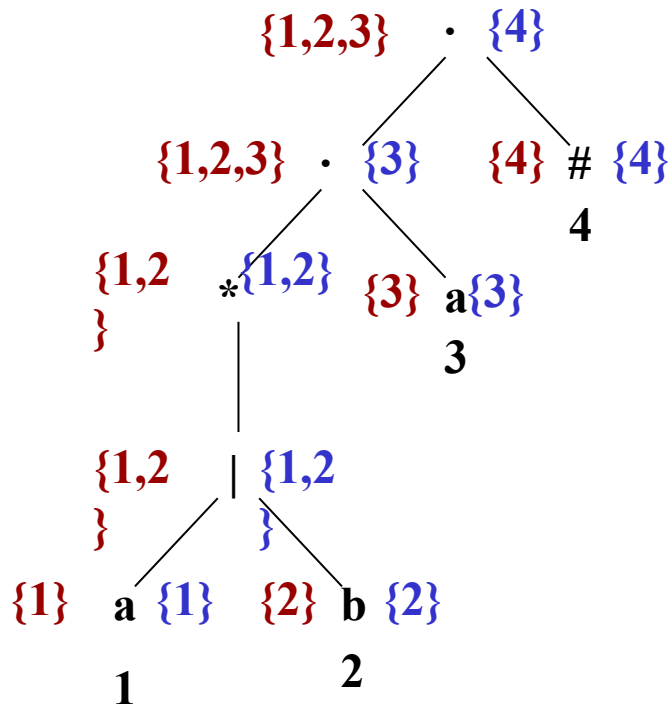
red – firstpos
blue – lastpos



Example - $(a \mid b)^* a \#$

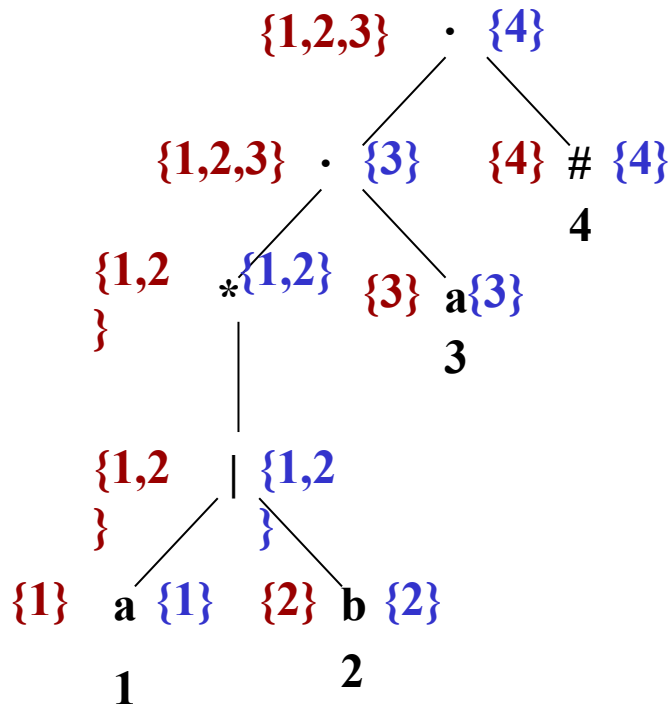


Example - $(a \mid b)^* a \#$



red – firstpos
blue – lastpos

Example - $(a \mid b)^* a \#$



red – firstpos
blue – lastpos

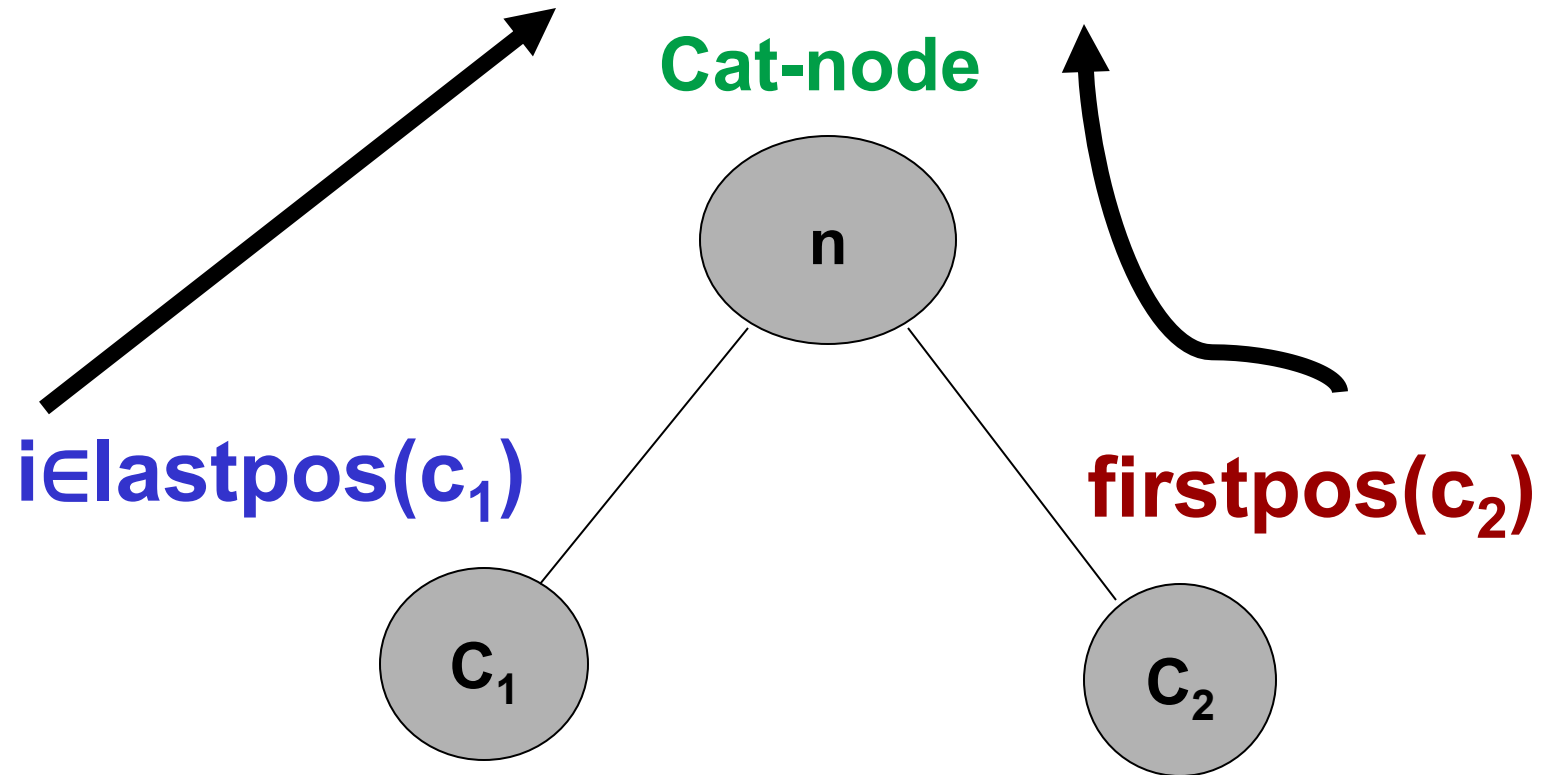
Then we compute followpos(i)

Two-rules define the function followpos

Rule for Concatenation-node:

If n is **concatenation-node** with
left child c_1 and right child c_2 and
 i is a position in **lastpos**(c_1)
then all positions in **firstpos**(c_2) are in **followpos**(i)

$$\text{followpos}(i) = \{ \text{firstpos}(c_2) \}$$



Two-rules define the function followpos

Rule for Concatenation-node:

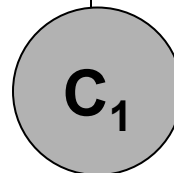
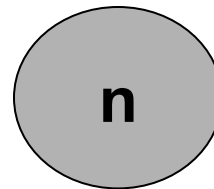
If n is **concatenation-node** with
left child c_1 and right child c_2 and
 i is a position in **lastpos**(c_1)
then all positions in **firstpos**(c_2) are in **followpos**(i)

Rule for star-node:

If n is a **star-node** and
 i is a position in **lastpos**(n)
then all positions in **firstpos**(n) are in **followpos**(i)

$\text{followpos}(i) = \{ \text{firstpos}(n) \}$

$\text{firstpos}(n)$ **Star-node** **$i \in \text{lastpos}(n)$**



Two-rules define the function followpos

Rule for Concatenation-node:

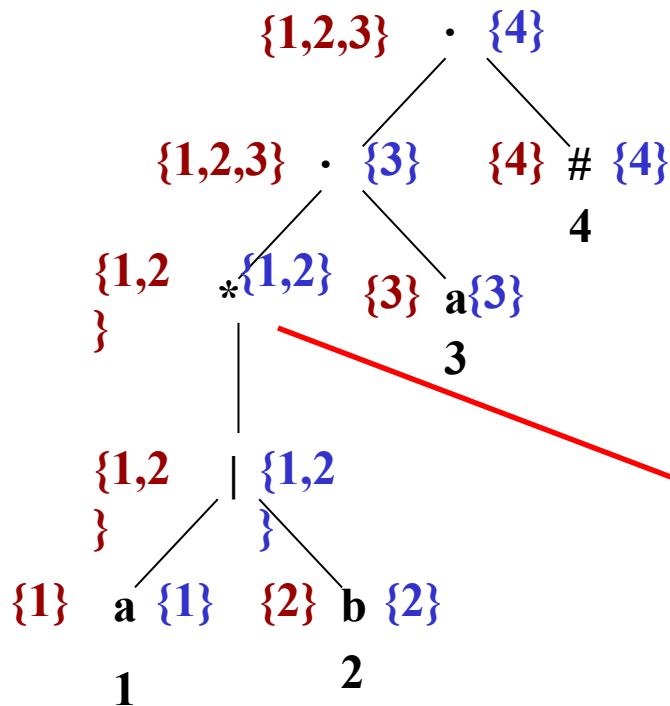
If **n** is **concatenation-node** with
left child **c₁** and right child **c₂** and
i is a position in **lastpos(c₁)**
then all positions in **firstpos(c₂)** are in **followpos(i)**

Rule for star-node:

If **n** is a **star-node** and
i is a position in **lastpos(n)**
then all positions in **firstpos(n)** are in **followpos(i)**

If firstpos and lastpos have been computed for each node, followpos of each position can be computed by making one depth-first traversal of the syntax tree.

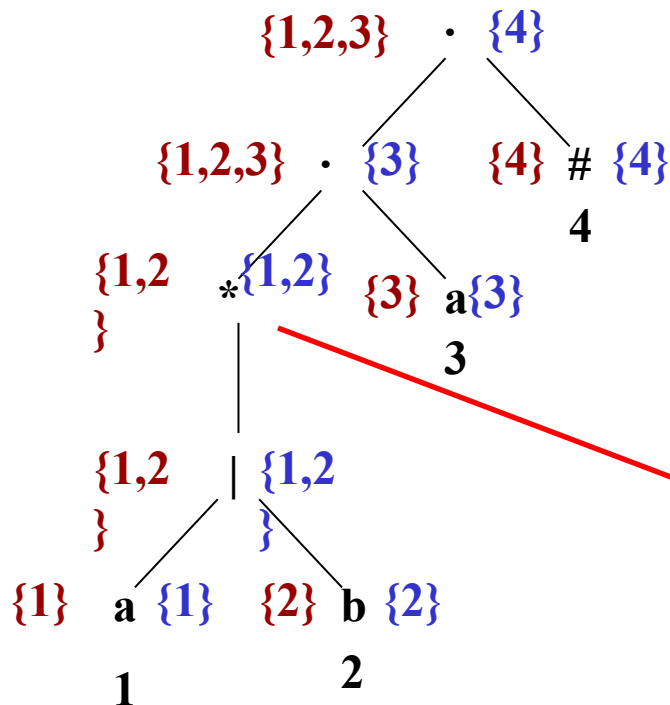
Example - $(a \mid b)^* a \#$



red – firstpos
blue – lastpos

n is a Star node: i is a position in **lastpos(n)**
then all positions in **firstpos(n)** are in
followpos(i)

Example - $(a \mid b)^* a \#$

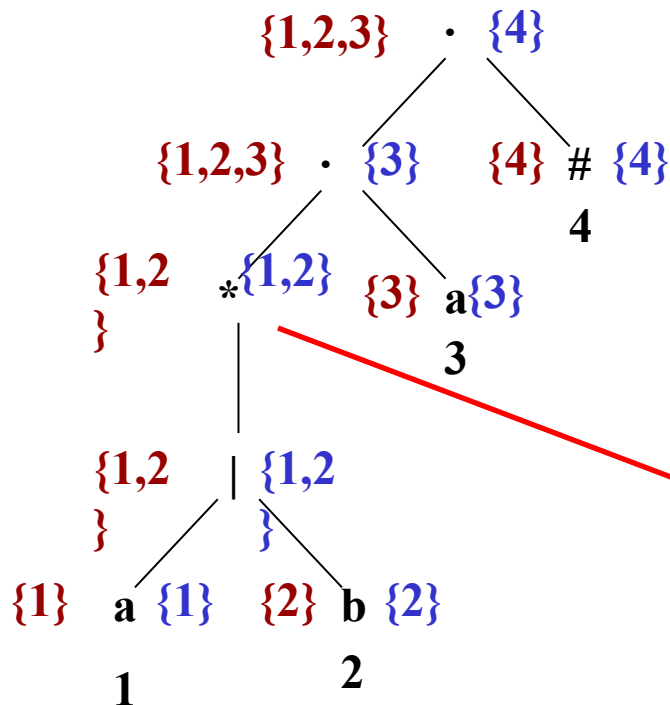


red – firstpos
blue – lastpos

n is a Star node: i is a position in lastpos(n)
then all positions in firstpos(n) are in
followpos(i)

followpos(1) =
followpos(2) =

Example - $(a \mid b)^* a \#$



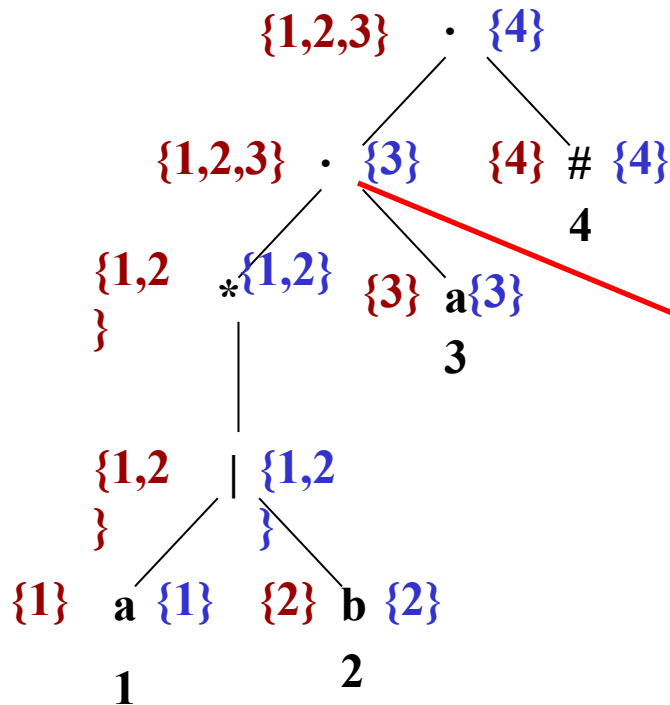
red – firstpos
blue – lastpos

n is a Star node: i is a position in **lastpos(n)**
 then all positions in **firstpos(n)** are in **followpos(i)**

followpos(1) = {1,2}

followpos(2) = {1,2}

Example - $(a \mid b)^* a \#$



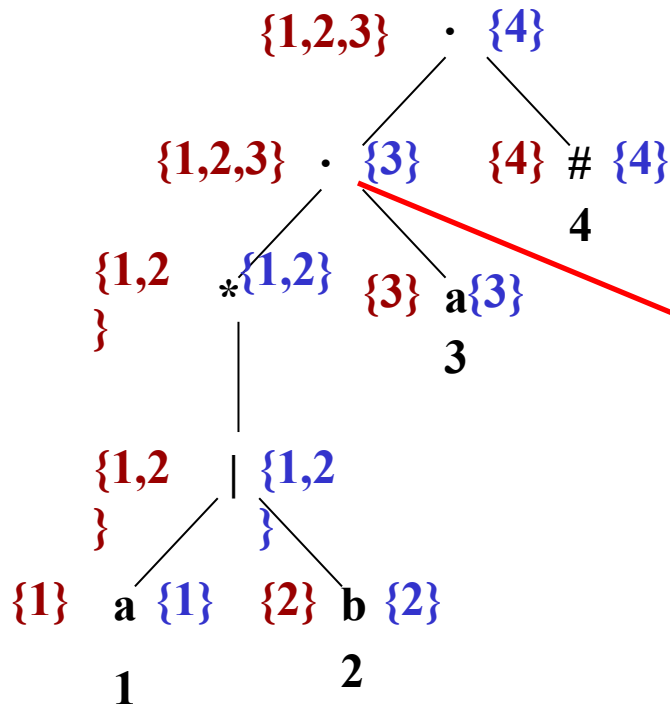
red – firstpos
blue – lastpos

n is a Cat node: i is a position in $\text{lastpos}(c_1)$
then all positions in $\text{firstpos}(c_2)$ are in
 $\text{followpos}(i)$

$\text{followpos}(1) = \{1,2\}$

$\text{followpos}(2) = \{1,2\}$

Example - $(a \mid b)^* a \#$



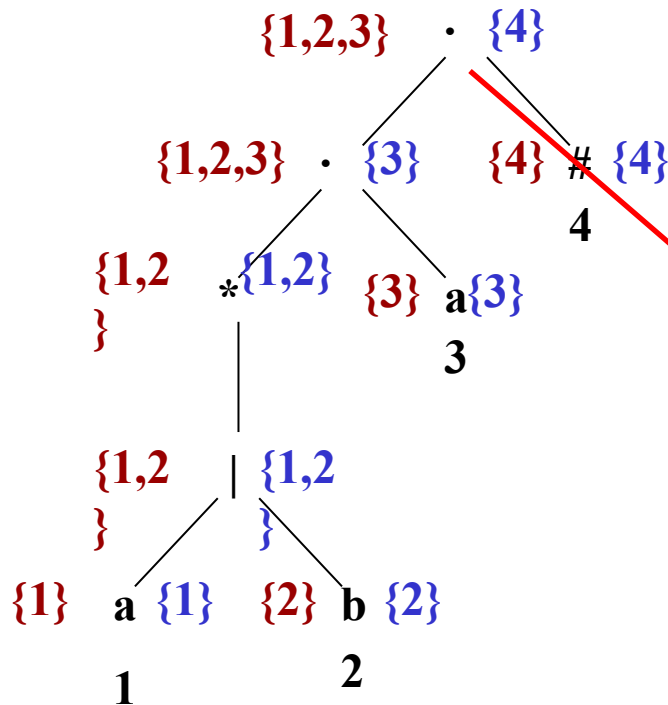
red – firstpos
blue – lastpos

n is a Cat node: i is a position in $\text{lastpos}(c_1)$
then all positions in $\text{firstpos}(c_2)$ are in
 $\text{followpos}(i)$

$\text{followpos}(1) = \{1,2,3\}$

$\text{followpos}(2) = \{1,2,3\}$

Example - $(a \mid b)^* a \#$



red – firstpos
blue – lastpos

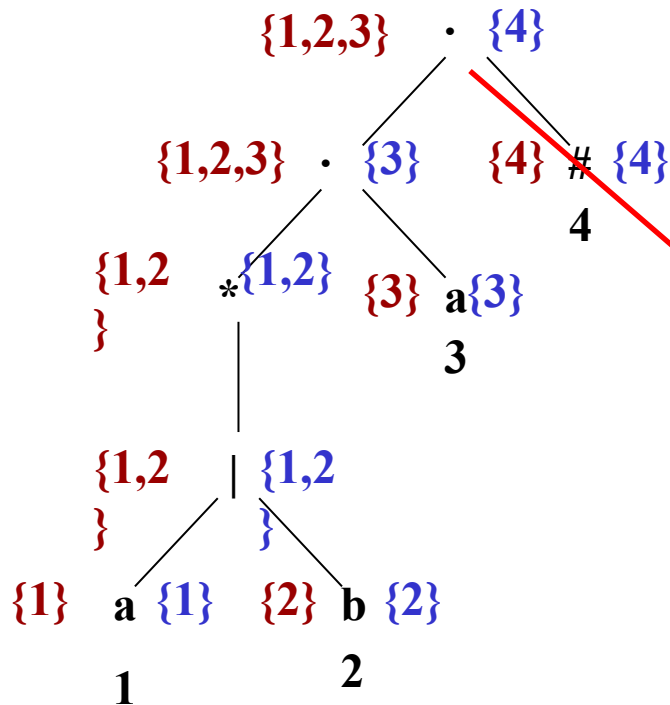
n is a Cat node: i is a position in $\text{lastpos}(c_1)$
then all positions in $\text{firstpos}(c_2)$ are in
 $\text{followpos}(i)$

$\text{followpos}(1) = \{1,2,3\}$

$\text{followpos}(2) = \{1,2,3\}$

$\text{followpos}(3) =$

Example - $(a \mid b)^* a \#$



red – firstpos
blue – lastpos

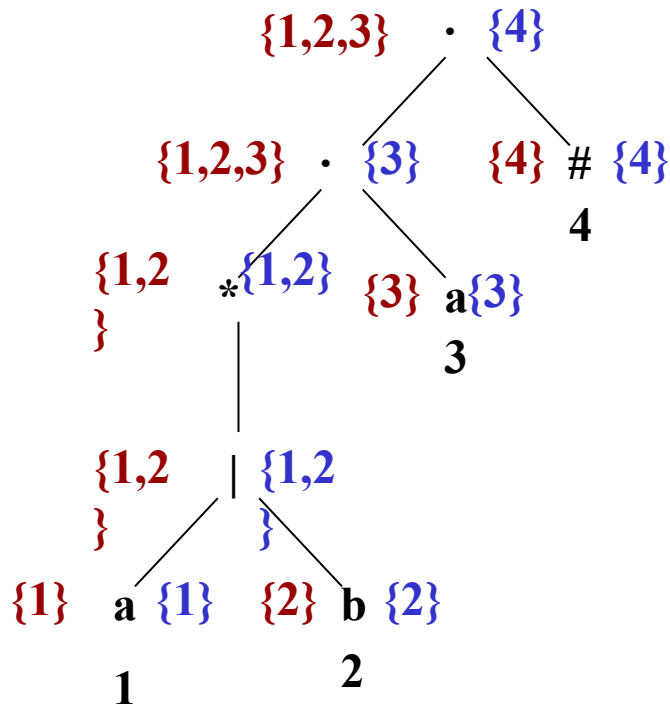
n is a Cat node: i is a position in $\text{lastpos}(c_1)$
then all positions in $\text{firstpos}(c_2)$ are in
 $\text{followpos}(i)$

$\text{followpos}(1) = \{1,2,3\}$

$\text{followpos}(2) = \{1,2,3\}$

$\text{followpos}(3) = \{4\}$

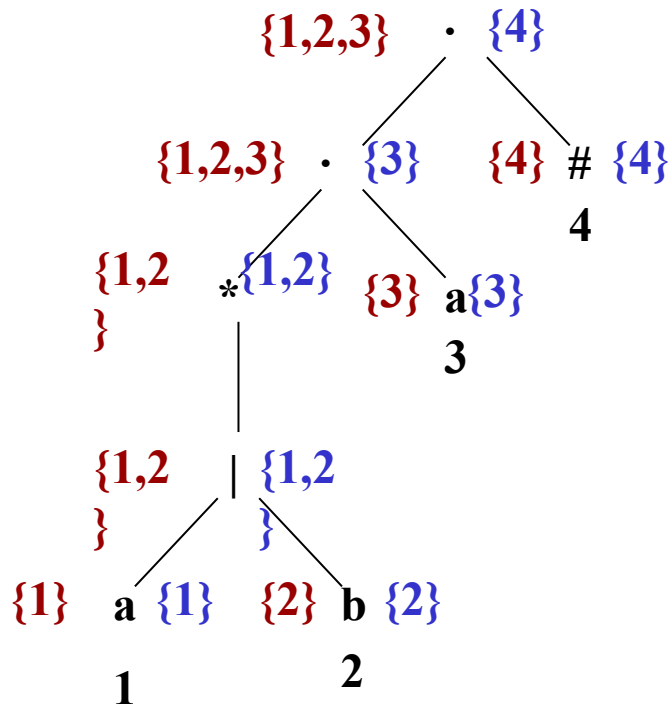
Example - $(a \mid b)^* a \#$



red – firstpos
blue – lastpos

followpos(1) = {1,2,3}
followpos(2) = {1,2,3}
followpos(3) = {4}
followpos(4) = {}

Example - $(a \mid b)^* a \#$



red – firstpos
blue – lastpos

followpos(1) = {1,2,3}
followpos(2) = {1,2,3}
followpos(3) = {4}
followpos(4) = {}

After we calculate follow positions, we are ready to create DFA for the regular expression.

Example -- (a | ϵ) b c*

1 2 3 4

followpos(1)={2} followpos(2)={3,4} followpos(3)={3,4} followpos(4)={}

$S_1 = \text{firstpos}(\text{root}) = \{1, 2\} = \text{start state}$

↓ mark S_1

$\text{move}(S_1, a) = \text{followpos}(1) = \{2\} = S_2$

$\text{move}(S_1, b) = \text{followpos}(2) = \{3, 4\} = S_3$

↓ mark S_2

$\text{move}(S_2, b) = \text{followpos}(2) = \{3, 4\} = S_3$

↓ mark S_3

$\text{move}(S_3, c) = \text{followpos}(3) = \{3, 4\} = S_3$

accepting states: $\{S_3\}$

